Consistency by Construction: Keeping a Multi-Project Agentic Ecosystem Correct as Projects and Agents Grow

Anonymous Author(s)
Affiliation withheld for double-anonymous review

***Abstract*—As agentic systems grow from single tasks into ecosystems of many projects, each with its own team of LLM agents, the hard problem becomes keeping the whole system consistent. Files must land where they belong, no project may depend on or overwrite another, there must be one source of truth, and no "done" may be false. We present Keel, a multi-project agentic ecosystem built and operated by one engineer with agents. Each of its eleven features was forced by a concrete failure, which we reconstruct from the system's own logs. Consistency is enforced *by construction*: one creation door checks placement and owner scope on every write, a database is the only registry, and an audit checks and heals at rest. We measure on an isolated copy of the production system and report ratios. A scripted workload demonstrates the mechanism: through the door, violations stayed at 0 for 1 to 10 concurrent projects, against 22–31% without it. The main evidence comes from real LLM agents. Our first measurement exposed a gap: files that agents wrote with their own tools bypassed the door, and correct placement fell from 67% to 17% as projects were added. We closed the gap inside the isolated copy with two changes that add no new rule. First, agents now start in their own project folder. Second, a pre-tool hook applies the existing owner-scope rule to the agents' own writes and sends shared writes through the door. Re-running the same experiment, correct placement became 100% at every scale, task success rose from 52/54 to 54/54, and no file landed outside its project. When agents were told to write into shared code and into another project, the hook blocked every raw attempt; the agents then used the door. The same work fixed two defects found by the owner's own tests. A regression suite of 32 checks passed with no regressions.**

I. Introduction

Agentic systems built from large language models (LLMs) are moving from single tasks to long-lived *ecosystems*: many projects, each with its own team of agents, sharing scripts, credentials, infrastructure and a common workspace. In this setting the hardest problem is not whether one agent can solve one task, but whether the *whole system stays consistent* as projects and agents are added: files written where they belong, no project silently depending on or overwriting another, one source of truth for what exists, and no "done" that is not true. Recent analyses of multi-agent frameworks report that most failures stem from organisational design, misalignment between agents and missing verification rather than from model capability [1].

This paper reports on Keel, a multi-project agentic ecosystem that its builder, a single engineer working with agents, developed and operates for real projects: e-commerce operations, CRM and e-mail platforms, fulfilment, continuous security monitoring, meeting intelligence and software delivery. Keel started as a learning exercise. A widely used multi-agent framework was judged unsuitable for consistent, safe use; the first project worked; the second project revealed *interference between projects*. Every subsequent component was forced by a concrete failure (Section II).

Central claim: Consistency and correctness can be made to *hold as the system scales* by construction: one creation door that validates placement and owner scope on every write, a database that is the only registry, and an audit that checks and heals at rest what bypassed the door.

Contributions:

1) An architecture of eleven features for multi-project agentic ecosystems. Each is traced to the failure that forced it and to its origin in established engineering practice: component reuse, middleware routing, message queues, webhooks, change-based deployment and conformance checking.
2) A failure timeline reconstructed from the system's own logs, documents and history (Table I).
3) A measurement study on an isolated copy of the production system, reporting ratios rather than absolute times. The scripted workload of RQ1 demonstrates the door's mechanism. The main evidence is RQ2, with real LLM agents running concurrently. The other studies cover the owner's own tests and the integration hub's queue, callback and live-sync behaviour.
4) A *before-and-after* study of one gap that the measurement exposed: the agents' own write tools bypassed the door. We closed it with two changes that add no new rule. We re-ran the same experiments, added an adversarial variant, and report both states side by side (Section III-A).

II. Origin: Each Failure Forced a Component

Keel began as a learning exercise. Its builder first evaluated a widely used multi-agent framework, judged it unsuitable for consistent and safe use, and deliberately chose a lean, deterministic dispatcher instead of a swarm (decision log, 21 June). The architecture was then grown one failure at a time. Table I lists every failure for which we found evidence in the system's own decision log, audit log, middleware-correction log, design documents or version history. Each row names the component that the failure forced into existence. Several

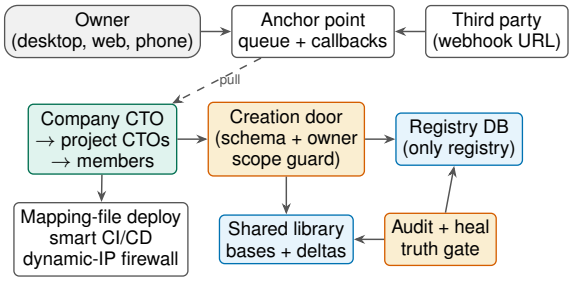


Fig. 1. Keel at a glance. Every created artefact passes one door; the audit checks what bypasses it.

components are independent re-discoveries of known patterns (Section VIII); we cite those patterns rather than claim them.

III. Architecture

Keel is a *multi-project agentic ecosystem*. A company-level CTO agent reasons over projects; each project has its own CTO and members (e.g., Fullstack, QA, DevOps, Fulfillment, Email, and twelve security agents); members work back and forth with the CTO, which re-assigns work when QA or the audit rejects a result. The engine is a coding agent (the Claude Code CLI) running in the project with terminal tools; Keel adds the organisation, the safety gates and the consistency machinery around it (Fig. 1). We describe the eleven features the owner identified, each with its origin.

F1 Component reuse (origin: React/React Native DRY): Shared scripts are treated as components and per-project deltas as their props [2], [3]: a total match points to the shared base, a near match is meant to store only a small delta that references the base, and a different job composes a different agent. This is the same idea as delta-oriented product lines [4], applied to the operational scripts that agents run. In the studied version, agents generated such deltas, but saving and registering them was incomplete (Section VII).

F2 Delta plus middleware semantic router (origin: back-end middleware): Reuse is decided by meaning: the agents read the middleware's guards and a registry view of the existing capabilities, and choose an existing base (reuse), a base plus delta, or new work. Placement and safety-relevant routing stay deterministic: a keyword floor sits under an optional local classifier, so they never depend on the model alone.

F3 Shared-script library with a context-indexed database: Before a project is created, an intake interview builds its features, its team and a readiness list that marks which resources already exist and which are missing; for security projects a blueprint provisions a complete security team. Agents propose reusing an existing capability, and name it, before building anything new. The database is the *only* registry: a parallel JSON registry is itself a violation. Every create, update and delete passes one *creation door* that validates placement against a canonical schema, redirects a misplaced file to its correct slot, prevents a project from writing into another project's folder (owner-scope guard), and writes the file and its database row together. In the studied version, changes applied after an owner's approval were registered only by the next audit.

F4 WhatsApp-style live sync: A change on any device propagates to all others. The Anchor point keeps a *durable* mirror, every message in a scope gets a monotonic sequence number assigned by one source, web and mobile clients pull deltas by sequence (a missing message is a visible gap), the desktop polls the pool, and a stale flag shows when the desktop is offline (Table I, 07-04).

F5 Queue at the Anchor point (origin: RabbitMQ): An external requester with a secure key submits a request; it is stored as *queued*. When the owner's machine is connected it pulls the queued requests in order of arrival, processes them, publishes and writes the status back. Pulling avoids any inbound connection to the owner's machine.

F6 Callback / return URL (origin: Stripe): The Anchor point posts the result to the requester's webhook and retries until it is acknowledged, so the contract is two-way: client-to-server and server-to-client [5].

F7 Cross-project agent reuse: Agents live in an organisation-wide roster in the database; a project *links* existing agents (CRM, fulfilment, e-mail-marketing and several security projects share agents) instead of cloning them.

F8 Dynamic-IP firewall (a feature): For operators without a static IP, deployment rotates the VM firewall and cloud security rule to the operator's current /32 address, replacing (never stacking) the previous one, so deployment stays possible without leaving a port open.

F9 Smart mapping-file deploy flow: A mapping file detects changed files by date and asks before updating; seed and database files are never touched automatically; a database update or rebase is asked separately, even on staging; after restart the flow verifies that real endpoints return real results and that newly seeded values are present.

F10 Smart CI/CD: Deterministic deployment "hands" are paired with an AI "brain" that probes the failing area (migration, build, ports, service, disk), consults a memory of learned faults, applies one safe fix and re-runs to verify, stopping for failures only a human can resolve.

F11 Audit layer: On a timer and on demand, an audit checks that files sit in the folder declared for them, that no project references another project's private files, that the registry mirrors the disk in both directions, and heals what it can (move, promote a duplicate to the shared library, re-point a reference, re-index). A separate *truth gate* rejects success claims that contradict the tools' own output; in the studied version it did not yet cover the reviewer's verdict at the end of a run (Section VII).

Also in the system: On-device Whisper speech-to-text and text-to-speech in English and Arabic; a Zoom meeting pipeline that turns live audio into a structured report; and phone and web control through the Anchor point. Connectors to SaaS platforms are plain API calls and are not a contribution.

TABLE I
FAILURE TIMELINE (2026). EVIDENCE PATHS ARE RELATIVE TO THE SYSTEM'S REPOSITORY. ONLY FAILURES WITH EVIDENCE ARE LISTED.

Date	Failure	Evidence	Component it forced
06-21	A widely used multi-agent framework was judged unsuitable for consistent, safe use at scale	decision log ("reject swarm scale, self-evolving rules")	Lean deterministic dispatcher, explicitly not a swarm; later the truth gate
06-21	The "brain" ran on a small local model instead of the intended coding agent	decision log ("CORRECTION")	The Claude Code CLI as the engine; the local model only as an offline fallback
06-22	An owner's "approved" did not attach to the engine's own pending proposal	decision log	Approval gate bound to the engine's session memory
06-22	Requests stopped mid-way: the dev reloader killed workers because the server wrote its own data files	decision log ("ROOT CAUSE")	Runtime data separated from watched code
06-22	Keyword routing sent a folder named `*-frontend` to the Frontend agent	decision log (two entries)	Semantic router over a deterministic keyword floor (F2)
06-23	Owner's end-to-end run: 33 pass, 1 fail (the planner hallucinated agents from "no backend")	decision log (FIX)	Negation-aware team inference; later the scoped organisation
06-23	The engine could read and run but not *safely* sync local code to the VM	decision log (NOTE)	Mapping-file deploy flow and smart CI/CD (F9, F10)
06-21	Dynamic home IP: SSH lock-out after every IP change	decision log; firewall-rotation module	Dynamic-IP firewall: one /32 rule, replaced, never stacked (F8)
07-04	Web and mobile could not reach the desktop without a static IP	isolation-state document	Anchor point with a pulled queue and callbacks (F5, F6)
07-04	Live sync broke across devices: the Anchor point kept its mirror in memory, so every restart or offline period wiped it	isolation-state document ("NEW ROOT CAUSE"); scope sequencer	Durable mirror, per-scope sequence numbers, stale flag (F4)
07-16	**Second-project interference**: files landed in the wrong place, the CTO duplicated agents it already had, rules were ignored	structuring plan ("Why this kills the failures you keep hitting")	One creation door, a schema, the database as the only registry (F3)
07-17–30	Measured violations, caught and healed: 18 misplaced, 16 duplicates, 15 foreign references, 5 external writes, 3 parallel registries; 33 wrong-folder writes redirected	audit log; middleware-correction log	Audit layer (F11); duplicates promoted to the shared library (F1)
07-19	First run of a 4-new-project journey: 0/4; five root causes (agents missing from cards, incomplete delete, one-directional audit, copies instead of reuse)	phase-7 results report	Bidirectional audit, clean delete, pointer-plus-delta reuse. Second run: 4/4
07-30	Connector keys were tracked in version control	commit history	Secrets kept out of git

A. Extending the door to the agents' own tools

The first run of RQ2 (Section V) showed that the door covered every write made *through the system*, but not files that a coding agent writes with its own editor and shell tools. We closed this gap inside the isolated copy with two changes. Neither adds a rule; both apply the rule F3 already states: an agent may *read* anywhere, including shared bases, may *write* only inside its own project (including delta files that point to a shared base), and may change shared code only through the door.

P2a: start in the own folder. Each project's agents start in that project's own working folder (its declared local path, otherwise its canonical code folder) instead of the repository root, and the absolute workspace path was removed from the instructions agents read. Relative writes therefore land inside the project.

P2b: a pre-tool hook. The agent runtime offers a hook that runs before every tool call and can refuse it with a message that the agent reads. The system already used this mechanism for read-only agents. For every edit or write call, and for the write targets of shell commands (redirections, `tee`, `mkdir`, `touch`, and the destination of `cp`/`mv`/`install`), the hook makes the same routing decision as the door, by calling the door's own router. The outcomes are: *allow* (own project, canonical slot, or temporary space); *redirect* (own project, wrong slot; the message names the correct path); or *block* (another project, the repository root, a system area, or shared code). For shared code the message names the agents' door, a small command that calls the same promotion routine the audit uses. Promotion moves the file to the shared library and leaves a pointer in the project, so the registry and the disk stay in step. The actor is set by the engine, not by the agent. The hook has three modes (off, watch, enforce).

We rolled P2b out in two stages. In *watch* mode the hook blocks nothing and logs every write it would redirect or block; any legitimate write in that log would have been routed through the door before switching on. In *enforce* mode it blocks.

IV. EVALUATION DESIGN

Our central claim is that *consistency and correctness hold as the system scales across projects and agents*. We ask:

- **RQ1** Does the creation door keep the workspace consistent as the number of concurrently active projects grows from 1 to 10, compared with agents writing directly? (A scripted workload that *demonstrates the mechanism*.)
- **RQ2** With real LLM agents running concurrently, how often do they write outside their own project or folder, does the audit catch it, and do tasks succeed? How does this change after extending the door to the agents' own tools (Section III-A)? (*The main evidence*.)
- **RQ3** Do the owner's own tests pass, before and after the changes? These are the lifecycle and adversarial tests and two LLM-driven tests.

- **RQ4** How do the Anchor point's queue, callbacks and live sync behave: where does the time go, are callbacks delivered under failures, and does the mirror survive restarts?

Setting and isolation: All experiments ran on a fresh, isolated copy of the production system with no credentials (all keys blank), every reference to the production folder rewritten, distinct ports, scheduled tasks disabled, and shims that block cloud and remote tools (Azure CLI, SSH, rsync, sshpass) for every process, including tools invoked by agents. The registry database is a private PostgreSQL cluster restored from a read-only dump of the production registry; the production database was only read. Every trial runs in a throw-away root with a freshly restored database. The machine is a MacBook Pro with an Apple M4 Pro (12 cores, 24 GB RAM, macOS 26.5). The owner's production instance and its scheduled security agents kept running; we record their CPU load at the start and end of each experiment and report *ratios* instead of absolute times, together with the CPU time and memory of the specific process tree under test.

RQ1 workload: $N \in \{1, 2, 5, 10\}$ projects, three agents each, all writing concurrently (12 writes per agent). The write mix follows the violation classes recorded in the system's history: 60% valid, 15% wrong sub-folder, 5% project-root files, 10% writes into *another* project, 5% duplicated utilities, 5% references to another project's private file. Condition DOOR sends every write through the real middleware; condition BYPASS writes to the requested path. We then run the real audit and heal. Five seeds per cell; 40 trials.

RQ2 workload: $N \in \{1, 2, 5, 10\}$ projects are created through the system's HTTP API, each with a Fullstack and a QA member, and all N team orchestrations start at once. Each project gets a small, verifiable task: a Python module and a plain-assert test, with a module name unique to the project. From a content hash of the whole root before and after, we measure files changed inside the own project, inside another project, and outside any project. We also record the audit's findings, whether the module exists, and whether *its test passes when we run it*. Cost and tokens come from the CLI's own accounting, and we record the share of wall time during which at least one model call was in flight. The model was fixed to a small model (`haiku`) for cost. Each N ran three times **before** the changes of Section III-A. After the changes we ran the watch stage (one repetition per N plus one extra at $N = 10$) and then repeated the **same** protocol, three repetitions per N, with the hook enforcing (**after**). We also ran an **adversarial** variant at $N = 5$: the task additionally asks each agent to copy its module into the shared library *and* into another project's code folder. It ran once with the hook off and once with it on. For this variant we read each agent's own transcript to classify how a file reached shared code.

RQ3: The owner's lifecycle test and adversarial test, unmodified, three repetitions each, before and after the changes. Two LLM-driven owner tests ran once before and once after: a multi-agent web build ("Revealer") and a journey of three projects created from prompts. Cloud steps were run locally, and one prompt that asks for a remote sandbox deployment was left out.

RQ4: Over real HTTP against the Anchor point's stand-in server: third-party clients submit M requests (50 with one client, 200 with four), a poller pulls, processes (synthetic, zero time, to isolate overhead) and publishes; a local receiver acknowledges callbacks, optionally failing 20% of them. Each request is time-tagged at *in, queued, pulled, processed, published* and *callback*. For live sync we push 20 pool updates and time push-to-visible, then restart the Anchor point and check the mirror. Three repetitions.

Regression suite: A suite of 18 checks was baselined before any change. It covers isolation invariants, compilation, live endpoints returning real data, the door's placement and owner-scope rules, the audit's detection classes, router determinism, the queue contract, and sequence contiguity. With the changes it grew to 32 checks: hook decisions (allow, redirect, block, door use and actor check, and two parser false positives found during the study), clean project deletion, and a resources section for every role. It was re-run after every change to the isolated copy. The final run passed 32/32 with no regression against the baseline.

V. Results

Environment: All runs share one MacBook Pro (M4 Pro, 12 cores). At the start of RQ1 the 1-minute load average was 3.4 and machine-wide CPU use 9%; the production instance used under 1% CPU during our runs, and its scheduled security agents last ran *before* the experiments started (13:42 and 13:46 UTC; experiments ran from 14:30), so they did not overlap any measurement. The before-and-after runs used the same machine and setup later the same day. Total model cost of the whole study: 16.46 USD.

A. RQ1: consistency at 1, 2, 5 and 10 concurrent projects

The scripted workload of RQ1 *demonstrates the mechanism*: it replays recorded error classes at fixed rates, so it shows what the door does with a known mix of bad writes. It is not evidence of how often real agents make them. That evidence is RQ2. Table II and Fig. 2 summarise 40 trials (5 seeds per cell). Through the door, *no* write ever landed in a wrong folder or in another project, at any scale: the violation rate is 0 with zero variance for $N = 1$ to 10 (36 to 360 concurrent writes by 3 to 30 agents). Without the door, 22% of writes violate the schema at $N = 1$ and 29–31% once there are other projects to interfere with; the number of writes that land inside *another* project grows with N (from 0 to 11% of all writes). The audit and heal step repairs every wrong-folder write in the bypass condition, but *not* the cross-project writes: a file written into another project's valid code folder is structurally legal at rest, so 11–12% of writes remain wrong after healing. Only the write-time owner-scope guard prevents this class. Through the door, the registry mirrored the disk exactly before any healing (lockstep 1.0), against 0.05 without the door; healing restores 1.0 in both cases. The residual audit findings with the door (0.07–0.10 per write) are content-level issues the door cannot

TABLE II
RQ1: VIOLATIONS PER WRITE (MEAN ± S.D., 5 SEEDS).

N	cond.	violations	after heal	cross-project	lockstep
1	no door	.217±.041	.000	.000	.05
1	door	**.000±.000**	.000	.000	1.00
2	no door	.289±.024	.106	.106	.05
2	door	**.000±.000**	.000	.000	1.00
5	no door	.307±.015	.118	.118	.05
5	door	**.000±.000**	.000	.000	1.00
10	no door	.308±.017	.108	.108	.05
10	door	**.000±.000**	.000	.000	1.00

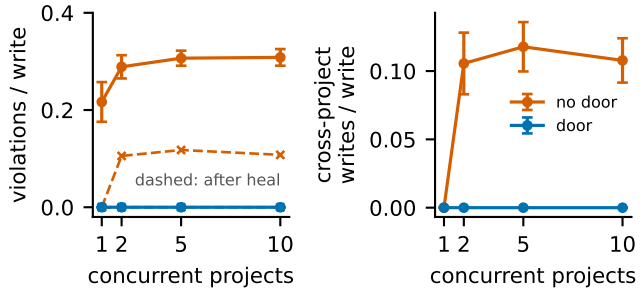


Fig. 2. RQ1. Left: violations per write, before (solid) and after (dashed) the audit heals. Right: writes landing in another project.

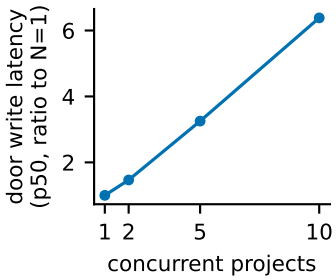


Fig. 3. RQ1. Median door write latency relative to one project.

TABLE III
RQ2 BEFORE AND AFTER EXTENDING THE DOOR TO THE AGENTS' OWN TOOLS (REAL LLM AGENTS, 3 REPETITIONS PER N, 54 TASKS PER STATE).

		success		own code slot		files outside proj./task	
N	tasks	before	after	before	after	before	after
1	3	1.00	1.00	.67	**1.00**	1.0	**0**
2	6	1.00	1.00	.50	**1.00**	1.5	**0**
5	15	.93	1.00	.40	**1.00**	1.2	**0**
10	30	.97	1.00	.17	**1.00**	1.7	**0**

Across all N: cross-project modules 1 → 0; writes into the main workspace 2 → 0.

see (references to another project's file, duplicated utilities); all were healed to zero.

Cost of the door: The door is 10–19× slower per write than a raw file write, and its median per-write latency grows 1.5×, 3.3× and 6.4× from 1 to 2, 5 and 10 concurrent projects (Fig. 3), because every write opens its own database connection under contention. In absolute terms this remains small relative to an agent step, which takes seconds to minutes, but it is the first scaling bottleneck we observed.

B. RQ2: real LLM agents, before and after

RQ2 is our main evidence. Table III shows the same 54-task experiment (1, 2, 5 and 10 concurrent projects, three repetitions) before and after the changes of Section III-A, side by side. Fig. 4 plots the two placement measures.

Before: Task success, measured by running each produced test ourselves, was 100%, 100%, 93% and 97% (52/54). Agents often wrote *outside the creation door*. Only 67%, 50%, 40% and 17% of tasks placed their artefacts in their own project's code folder; the rest wrote to a top-level folder of the root, the agent's starting directory. The audit flagged every such stray top-level path (flag ratio 1.0 in every run that had one), but it does not move them automatically. Two further failures appeared at scale. In one repetition at $N = 10$, five task names were reused, and four pairs of projects overwrote each other's files in the shared stray folder. In another, an agent wrote one module into another project's folder, and two agents followed the absolute repository path in the system's instructions and wrote into the main workspace instead of their throw-away root.

Watch stage: With P2a in place and the hook only logging (28 tasks), the hook was invoked on 58 tool calls in the traced run and would have redirected or blocked none of them. All artefacts were in their own code slot. No legitimate write needed routing through the door, so the hook was switched on unchanged.

After: With the hook enforcing, every task at every N placed its artefacts in its own code slot (1.00), success was 54/54, and no file was written outside a project, into another project, or into the main workspace. The hook allowed every call in this workload and blocked none: starting agents in their own folder (P2a) is what fixed placement for well-behaved agents. Cost per task changed by a factor of 0.79, 0.90, 0.80 and 0.75 relative to the same N before. Its growth with N (ratio to $N = 1$: 1.13, 1.19, 1.06) stayed similar to before (0.99, 1.18, 1.13).

Adversarial variant: Well-behaved agents never tested the hook's blocking, so we asked agents to break the rule (Table IV). With the hook off, all five agents wrote their module directly into the shared library and into another project's code folder. Five modules landed in other projects. The audit flagged none of the shared-library writes, because a file in the shared library is legal at rest. With the hook on, every raw attempt was refused: 9 shell writes into shared code and 5 into another project. No module landed in another project. The agents read the refusal message and promoted their module through the door instead (5/5, confirmed in their transcripts). Promotion leaves a pointer in the project and registers the file, so the shared library changed only through the audited route. One agent's own test failed in this run (4/5).

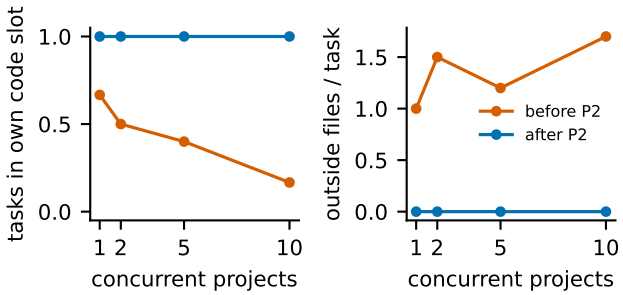


Fig. 4. RQ2 before (red) and after (blue). Left: share of tasks with all artefacts in the own code slot. Right: files written outside any project per task.

TABLE IV
ADVERSARIAL VARIANT ($N = 5$): EACH AGENT IS ASKED TO COPY ITS MODULE INTO THE SHARED LIBRARY AND INTO ANOTHER PROJECT.

	hook off	hook on
raw writes into shared code	5/5 agents	0 (9 blocked)
modules landed in another project	5	**0** (5 blocked)
shared modules via the door	0/5	**5/5**
audit flags on shared writes	0	n/a (door)
task tests passing	5/5	4/5

What the adversarial run taught us about the hook: Two weaknesses surfaced, and both were fixed without new rules. In the first hook-on run some agents set the actor variable themselves. They set it to their own project, so no harm resulted, but it showed that the door could have been called under another project's name. The hook now refuses a door call made under a foreign actor, and a door call no longer excuses another raw write chained in the same command. A later owner test exposed two false positives in the hook's shell parser: a > inside a quoted `sed` expression, and code fed to Python through a here-document, were read as write targets. The parser is now quote- and here-document-aware. All four cases are regression checks. The final adversarial numbers in Table IV come from a re-run with the fixed hook. The first run gave the same blocking result (15 blocked, 0 cross-project modules).

C. RQ3: the owner's own tests, before and after

Table V summarises the owner's four tests. The lifecycle test creates four projects of different shapes through the door, monitors them and deletes them. It was all green in 3/3 runs both before and after the changes. Each creation had zero placement errors and zero violations. The deploy-shaped project reused the shared deploy capability through a pointer, with no second deploy script. Deletion returned every registry count to its exact baseline. The adversarial test injects every violation type and requires the system to correct it on disk and in the database. It passed 32/32 checks in 3/3 runs, before and after. Before the changes it also *failed* once, correctly: it reported the two files that agents had written into the main workspace as cross-project duplicates and refused to declare a clean baseline.

TABLE V
RQ3: THE OWNER'S OWN TESTS, BEFORE AND AFTER THE CHANGES.

test	before	after
lifecycle (4 shapes, 3 runs)	all green 3/3	all green 3/3
adversarial (32 checks, 3 runs)	32/32 ×3	32/32 ×3
Revealer pre-flight	blocked (shipment-tracking res.)	**passes**
Revealer scorecard	not run	5/10 (no creds)
prompt journey: clean delete	0/3 (1–2 rows left)	**2/3** (+1 system row)
prompt journey: team coverage	1/3	1/3
regression suite	18/18	**32/32**, 0 regressions

The two LLM-driven tests revealed two defects, which we fixed in the isolated copy.

- **Missing role resources.** The most recently added role (shipment-tracking) had no resources-and-dependencies section in the role catalogue, so the multi-agent "Revealer" build stopped at its own pre-flight gate. We added the catalogue entry, with its dependencies taken from the role's own definition. After the fix, the pre-flight passed and the build ran. The CTO agent then stopped truthfully, before writing any application code: the isolated copy has no credentials (by design), and its rules forbid reporting numbers without a credentialed source. The scorecard therefore moved from "blocked at pre-flight" to 5/10 rows (team derivation, planning, CTO present, truthfulness, completed in time). The five build rows cannot pass without credentials. The hook blocked nothing in this run.
- **Leftover registry rows.** In the prompt journey (three projects created from prompts), deletion left one or two registry rows behind in 3/3 projects: the files of the project's conversation scopes. The door's delete now removes those scope folders and their rows. After the fix, 2/3 deletions were exactly clean. The third showed one extra row, which is not the project's: the system's own audit log, registered for the first time on the freshly restored database while that project was running. No row owned by any deleted project remained.

The journey's other findings did not change: the derived team covered the required capabilities in 1/3 projects, and 2/3 produced output. Both depend on how teams are derived from the prompt, which we did not change. After this test we also found agent processes of already-deleted projects still running. We stopped them and record this as a proposal.

D. RQ4: queue, callbacks and live sync

Fig. 5 shows where end-to-end time goes for a request through the Anchor point with zero processing time. Waiting in the queue for the next pull is 97–99% of the time; submission, publication and callback together are 1–3%. Four concurrent clients give 2.1× the throughput of one. With 20% of callbacks failing, the Anchor point's own retry sweep delivered 199/200 within the run and the last one at the next sweep (an at-least-once delivery, seen as a duplicate in the following run), so eventual callback success was 100% in all three repetitions, at the price of retry waits dominating (66–76% of time). For live sync, a pushed update was visible to a delta reader in under

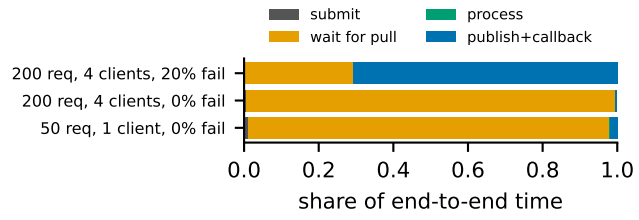


Fig. 5. RQ4. Share of end-to-end request time per stage (3 repetitions).

0.1% of the poller's push interval; after a restart the mirror and all 20 sequenced messages were present, and a delta request from sequence 15 returned exactly 16–20.

VI. Discussion

Prevention and detection are both needed: RQ1 separates two mechanisms that are often conflated. The audit can repair everything that is wrong *in form* (misplaced files, unindexed rows, duplicates, foreign references), but a write into another project's legitimate folder is wrong only *in ownership*, and at rest it looks legal; 11–12% of writes stay wrong after healing without the door. The owner-scope guard at write time is what keeps interference at zero as projects are added. This mirrors the history in Table I: the audit came first, and interference between projects forced the door.

The agent's own pen, before and after: Before the changes, RQ2 showed where consistency was not yet by construction: files that a coding agent wrote with its own tools did not pass through the door. Placement fell from 67% to 17% as projects were added, and at ten projects agents overwrote each other in a stray folder. The audit saw every stray path, but flagging is not preventing. The fix needed no new rule. Starting agents in their own folder made well-behaved agents place everything correctly (100% at every N). The pre-tool hook made the rule hold for agents that try to break it: every raw write into shared code or into another project was refused, and the agents fell back to the door. Each layer covers a different case. P2a is enough for agents that cooperate; P2b is needed for agents that do not; and only the door keeps the registry in step when shared code does change. The audit stays as the at-rest check for what the hook cannot see, for example a file opened for writing from inside a script.

Watch first, then enforce: The staged rollout cost one repetition and gave a concrete answer: with P2a in place, no legitimate write would have been blocked. The adversarial runs were what exercised the hook. They also exposed two weaknesses that ordinary work never would have: a self-declared actor and a door call that excused a chained raw write. That argues for including adversarial tasks in any evaluation of guard rails for agents.

Consistency did not cost scale: Through the door the violation rate stayed at zero from 1 to 10 projects, cost per task stayed flat, and the system's own process tree was negligible next to model time. The measurable price is per-write latency under contention (6.4× at ten projects), which a connection pool would address without touching the schema.

Pulled queues trade latency for reachability: Almost all of a request's time is spent waiting to be pulled. That is the deliberate cost of never opening an inbound port on the owner's machine; callbacks with retries make the contract reliable, and receivers must tolerate an occasional duplicate.

Owner tests as instruments: The owner's own tests were the most sensitive instruments we had. Without being designed for our study, they caught two files that agents had written into the main workspace, an incompletely registered new role, and leftover registry rows. After the fixes they also exposed two hook false positives and agent processes that outlive their deleted project.

A. Proposals (not applied)

The owner's rule is that no change may harm structure, consistency, database design or connections. P2 and the two defect fixes were therefore implemented and measured only in the isolated copy. They are handed to the owner as a promotion package of patches that apply cleanly to the production files, and they are not applied. The remaining proposals are written only: (P1) move the terminal bridge's runtime files outside the version-controlled folder; (P3) pool database connections in the door; (P4) for *finding* reusable scripts only, add a sentence-embedding similarity [6], [7] as a second ranking signal next to keywords, while structural enforcement stays exact and rule-based; to be benchmarked on owner-labelled requests; (P5) a retrieval pipeline (recursive or semantic chunking, a small embedding model, a local vector index, top-k with cross-encoder re-ranking) for the one project that writes stories from large, stable reference PDFs, and nowhere else; (P6) send the request identifier as an idempotency key with every callback; (P8) when a project is deleted, also stop its running agent processes; (P9) derive the prompt journey's teams from required capabilities before adding newly registered roles; (P10) make the few modules that write relative to the current directory use the workspace root.

VII. Threats to Validity

Construct: Our consistency metrics are the system's own checks (schema placement, owner scope, foreign references, registry lockstep) plus an authorship marker for cross-project writes; violations outside these classes are not counted. Task success for the LLM runs is a module plus a passing test that *we* execute, not the agents' claims.

Internal: The scripted workload (RQ1) replays the error classes observed in the system's history with fixed proportions. It demonstrates the mechanism, not the rate at which real agents err; RQ2 measures that. The before-and-after comparison changes one thing at a time only in part: P2a and P2b were measured together after the watch stage, but the hook blocked nothing in the normal workload, so the placement gain is attributable to P2a. The adversarial variant is a single run per condition at $N = 5$. The hook's shell parser is best-effort, and writes from inside a script are left to the audit.

All runs used the same machine while the owner's production instance and scheduled security agents were active; we record their load and report ratios rather than absolute times. Cloud and remote tools were blocked by design, so deployment-to-VM steps of the owner's tests were run locally (Section IV). The registry database is a private copy restored from a read-only dump. Two weaknesses of the studied version lie outside our consistency metrics: per-project deltas were generated but not always saved and registered, and the truth gate did not yet cover the reviewer's verdict, so a run could report success while its review asked for a fix. Neither affects the reported placement, violation or task-success figures, which are taken from disk, from the system's own checks and from tests that we run ourselves.

External: This is one system, built mainly by one engineer with agents, on one laptop. Results may not transfer to other organisations; the mechanisms (one door, registry-only database, audit) are, however, framework-independent.

Reliability: We repeated every run: 5 seeds for RQ1; 3 repetitions per N before and after for RQ2; 3 repetitions for the owner's scripted tests and for RQ4. The LLM-driven owner tests ran once per state. One invalid run is excluded and reported: our own regression run reset the database under it. A replication package with all scripts, raw results and the decision log is provided.

VIII. Related Work

Why multi-agent systems fail: The MAST study analyses execution traces of seven multi-agent frameworks and attributes failures to specification issues, inter-agent misalignment and task verification [1]; it argues that many failures are organisational rather than model limitations. Keel is a practitioner's answer to the organisational part: structure is enforced by construction rather than requested in prompts. Multi-agent frameworks such as AutoGen, MetaGPT and ChatDev [8]–[10] coordinate roles within one task; they do not govern a shared, multi-owner workspace.

Reuse of agent skills (F1–F3, F7): Skill and workflow libraries let agents reuse code they wrote before [11]–[14]; SkillOps manages such a library as a self-maintaining ecosystem [15]; tool retrieval selects among many APIs [16]; plan and semantic caches reuse earlier answers [17]–[19]; case-based reasoning frames reuse in general [20], [21]. Keel's base-plus-delta discipline is the component idea [2], [3] and delta-oriented programming [4], re-discovered for agent scripts; what it adds is a single creation door and a database that is the only registry across *many owners' projects*.

Routing (F2): Model routers trade cost for quality [22]; Keel's agents choose between reuse, delta and new work by reading a registry view, while placement stays rule-based; sentence embeddings are only proposed (Section VI-A).

Sync, queueing and callbacks (F4–F6): Local-first software and CRDTs give strong convergence guarantees [23]; Keel uses the simpler combination of a durable mirror and per-scope sequence numbers in the spirit of logical clocks [24]. Its pulled queue is a minimal message broker [25], and its callbacks follow industry webhook practice with retries until acknowledgement [5].

Deployment and CI (F8–F10): Zero-trust access [26] removes reliance on network location; Keel's dynamic-IP firewall instead keeps a single current address allowed for operators without a static IP. Change-based deployment relates to regression test selection by file dependencies [27], and the self-healing deploy agent to LLM-based repair agents [28].

Conformance and guardrails (F11): Reflexion models check source against an intended architecture [29]; Keel's audit applies the same idea to the artefacts that agents create, and heals deviations. Runtime enforcement for agents [30], programmable rails [31] and capability-based designs [32], [33] constrain actions. Keel's pre-tool hook (Section III-A) is a runtime check of this kind, but it adds no rule language of its own: it calls the same router as the creation door. The truth gate addresses part of the verification failures reported by MAST.

IX. Conclusion

A multi-project agentic ecosystem stays consistent as it grows when consistency is a property of the architecture rather than a request to the model. In Keel, one creation door with an owner-scope guard, a database that is the only registry, and an audit that heals at rest kept violations at zero from one to ten concurrent projects in a scripted demonstration. Removing the door produced interference that no later audit could undo. With real agents, the first measurement showed the gap: the agents' own write tools sat outside the door. We closed it without adding a rule. Agents now start in their own folder, and a pre-tool hook applies the existing owner-scope rule and sends shared changes through the door. Correct placement rose from 17–67% to 100%, and agents told to break the rule were stopped and used the door instead. Every component in Keel exists because something broke. We hope the failure timeline, and the before-and-after evidence, are as useful to other builders as the components themselves.

References

[1] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why do multi-agent LLM systems fail?" in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025, arXiv:2503.13657.

[2] Meta Open Source, "Thinking in React," https://react.dev/learn/thinking-in-react, accessed 2026-10-06.

[3] M. D. McIlroy, "Mass produced software components," in *Software Engineering: Report of a Conference Sponsored by the NATO Science Committee*, 1968, pp. 138–155.

[4] I. Schaefer, L. Bettini, V. Bono, F. Damiani, and N. Tanzarella, "Delta-oriented programming of software product lines," in *Software Product Lines: Going Beyond (SPLC)*, ser. LNCS, vol. 6287. Springer, 2010, pp. 77–91.

[5] Stripe, "Receive Stripe events in your webhook endpoint," https://docs.stripe.com/webhooks, accessed 2026-10-06.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[7] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[8] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversations," in *Conference on Language Modeling (COLM)*, 2024.

[9] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[10] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 15 174–15 186.

[11] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research*, 2024.

[12] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig, "Agent workflow memory," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 267, 2025.

[13] Z. Z. Wang, A. Gandhi, G. Neubig, and D. Fried, "Inducing programmatic skills for agentic tasks," in *Conference on Language Modeling (COLM)*, 2025.

[14] S. Yu, G. Li, W. Shi, and P. Qi, "PolySkill: Learning generalizable skills through polymorphic abstraction for continual learning," in *International Conference on Learning Representations (ICLR)*, 2026.

[15] H. Pu, X. Song, and L. Zhao, "SkillOps: Managing LLM agent skill libraries as self-maintaining software ecosystems," *arXiv preprint arXiv:2605.13716*, 2026.

[16] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[17] Q. Zhang, M. Wornow, G. Wan, and K. Olukotun, "Agentic plan caching: Test-time memory for fast and cost-efficient LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[18] F. Bang, "GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings," in *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, 2023, pp. 212–218.

[19] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar, "MeanCache: User-centric semantic caching for LLM web services," in *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2025.

[20] A. Aamodt and E. Plaza, "Case-based reasoning: Foundational issues, methodological variations, and system approaches," *AI Communications*, vol. 7, no. 1, pp. 39–59, 1994.

[21] K. Hatalis, D. Christou, and V. Kondapalli, "Review of case-based reasoning for LLM agents: Theoretical foundations, architectural components, and cognitive integration," *arXiv preprint arXiv:2504.06943*, 2025.

[22] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs from preference data," in *International Conference on Learning Representations (ICLR)*, 2025.

[23] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proceedings of the ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, 2019, pp. 154–178.

[24] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[25] S. Vinoski, "Advanced message queuing protocol," *IEEE Internet Computing*, vol. 10, no. 6, pp. 87–89, 2006.

[26] R. Ward and B. Beyer, "BeyondCorp: A new approach to enterprise security," *;login: The USENIX Magazine*, vol. 39, no. 6, pp. 6–11, 2014.

[27] M. Gligoric, L. Eloussi, and D. Marinov, "Practical regression test selection with dynamic file dependencies," in *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, 2015, pp. 211–222.

[28] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025.

[29] G. C. Murphy, D. Notkin, and K. J. Sullivan, "Software reflexion models: Bridging the gap between source and high-level models," in *Proceedings of the 3rd ACM SIGSOFT Symposium on Foundations of Software Engineering (FSE)*, 1995, pp. 18–28.

[30] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE)*, 2026, pp. 2938–2950.

[31] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2023, pp. 431–445.

[32] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," in *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2026.

[33] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.